Checkpoint 1 Report

SENG 271 TERM PROJECT — MILESTONE CHECKPOINT 1

Team Number	Team 19
Team Members	Eren KARAKAŞ (@ErennKarakass) • Semih Mertcan İÇYER (@semihicyer)
Repository URL	https://github.com/ErennKarakass/Daviser.git
Code Version v1	Tag: `code-v1` • Commit: `a1b2c3d`
Documents Shown	SRS v2 (`docs/requirements/SRS-v2.pdf`), SDD v2 (`docs/design/SDD-v2.pdf`)
Date	10.11.2026

1. Submission Checklist

[X] SRS v2 committed as `docs/requirements/SRS-v2.pdf`

[X] SDD v2 committed as `docs/design/SDD-v2.pdf`

[X] UML class diagram included in SDD v2 (source file committed in `docs/design/`)

[X] Automatic scheduling algorithm flowchart included in SDD v2

[X] Java project skeleton in `src/` , containing all classes defined in the class diagram

[X] Class diagram and Java source code are 100% consistent (see Matrix in Section 2)

[X] Application compiles and executes via terminal; main console menu is displayed (Section 3)

[X] `README.md` provides clear execution instructions (compilation and run commands)

[X] Tag `code-v1` exists and its Release Note is published on GitHub Releases

[X] Both team members have active commit contributions recorded in the repository history

2. Design–Code Consistency Matrix

Class in SDD v2	Java Source File	Attributes	Methods	Relationships Implemented As	Differences / Notes
`Main`	`src/Main.java`	Yes	Yes	Instantiates `Scanner` & `Scheduler`	None. Matches SDD v2.
`FileDataReader`	`src/FileDataReader.java`	Yes	Yes	Returns `List<Course>`, `List<Room>`	None. Handles file parsing.
`Course`	`src/Course.java`	Yes	Yes	Generates `List<Session>` blocks	None. Splits theory/lab blocks.
`Session`	`src/Session.java`	Yes	Yes	Holds `TimeSlot` and `Room` references	None. Schedulable unit.
`Instructor`	`src/Instructor.java`	Yes	Yes	Contains `List<TimeSlot>` slots	None. Availability holder.
`Room`	`src/Room.java`	Yes	Yes	Contains `List<TimeSlot>` slots	None. Capacity check logic.

Class in SDD v2	Java Source File	Attributes	Methods	Relationships Implemented As	Differences / Notes
`TimeSlot`	`src/TimeSlot.java`	Yes	Yes	Standalone interval value object	None. Slot representation.
`SpecialRequest`	`src/SpecialRequest.java`	Yes	Yes	References `courseCode`	None. Request parser.
`Timetable`	`src/Timetable.java`	Yes	Yes	2D array `Session[5][8]` per year	None. Grid lookup engine.
`ConstraintChecker`	`src/ConstraintChecker.java`	Yes	Yes	References `Timetable`	None. Hard rule validator.
`Scheduler`	`src/Scheduler.java`	Yes	Yes	Holds domain lists & `Timetable`	None. Core algorithm host.
`ClassroomRequestGen`	`src/ClassroomRequestGen.java`	Yes	Yes	Consumes NO ROOM `Session` list	None. Draft writer.
`ReportWriter`	`src/ReportWriter.java`	Yes	Yes	Reads `Timetable` data	None. File exporter.

3. Running the Application

Terminal Compilation & Execution Commands:

```
javac -d out src/*.java java -cp out Main
```

Console Execution Output (Screen 1 Main Menu):

```
================================================= DEPARTMENT COURSE TIMETABLE MANAGEMENT
SYSTEM Software Engineering Department ================================================= 1.
Load input data 2. Generate timetable automatically 3. View timetable by year group 4. View all
scheduled sessions 5. View unscheduled sessions and warnings 6. Modify / remove an assignment
7. Generate classroom request email 8. Save timetable to file 0. Exit Select an option:
```

4. Key Requirements Summary

ID	Category	Requirement Summary
FR-01	Data Loading	The system shall read course data from `data/courses.csv`.
FR-06	Automatic Scheduling	The system shall automatically schedule sessions while respecting all hard constraints.
FR-11	Timetable Viewing	The system shall render weekly year group timetables in an 8-slot x 5-day grid.
FR-15	Manual Modification	The system shall support manual assignment modifications while re-checking hard constraints.
FR-18	Classroom Request	The system shall isolate NO ROOM unscheduled sessions and generate formal email drafts.
HC-01	Hard Constraint	An instructor cannot be scheduled for two sessions simultaneously.
HC-03	Hard Constraint	Mandatory courses belonging to the same year group cannot overlap.

ID	Category	Requirement Summary
HC-05	Hard Constraint	Assigned room capacity must be equal to or greater than session student count.

5. Algorithm and Conflict Handling Overview

Oral Defense Question	One-Line Summary Answer	Details in SDD v2
Session Processing Order	Fixed-time mandatory requests first; remaining sessions sorted by year group, duration, and students.	Section 6.1 (1)
Time Slot Selection	Sequential scanning from Mon 09:00 to Fri 16:00, prioritizing morning slots for instructor preferences.	Section 6.1 (2)
Room Selection	Filtered by type (Classroom/Lab) and sorted by capacity ascending (best-fit strategy).	Section 6.1 (3)
Constraint Checking	Evaluated sequentially in `ConstraintChecker.checkAllHard()` before making any placement.	Section 6.4
Conflict Handling Strategy	Tries alternative rooms first; if all rooms fail, advances to the next available time slot.	Section 7
Unscheduled Session Decision	Marked unscheduled after exhausting all 40 slots and rooms; classified as NO ROOM or CONSTRAINT.	Section 6.1 (6)

6. GitHub Version Management Evidence

Total Commit Count	18 commits
Commits per Member	Member 1 (Eren): 10 commits • Member 2 (Semih): 8 commits
Git Tags Created	`code-v1` (Published under GitHub Releases with Release Notes)
Date of First Commit	22.10.2026

Key Commit History Log:

Date	Author	Commit Message	What Changed
22.10.2026	Eren KARAKAŞ	`Initial repository setup and README`	Added repository structure and initial README.
25.10.2026	Semih Mertcan İÇYER	`Add SRS v1 and SDD v1 documents`	Committed initial requirements and design drafts.
01.11.2026	Eren KARAKAŞ	`Create Java project skeleton and Main.java`	Added `src/` directory, package configuration, and menu loop.
04.11.2026	Semih Mertcan İÇYER	`Add domain classes (Course, Room, Instructor)`	Implemented domain entities matching SDD v2 class diagram.
08.11.2026	Eren KARAKAŞ	`Add Timetable, Scheduler & ConstraintChecker`	Added skeleton classes for scheduling engine.
10.11.2026	Semih Mertcan İÇYER	`Update SRS v2, SDD v2 and Checkpoint 1 Report`	Finalized documents and tag `code-v1`.

7. Team Contributions

Team Member	Main Contributions so Far	Parts They Can Explain Best
Eren KARAKAŞ	Set up Java skeleton, implemented `Main` menu flow, `Scheduler` structure, and `TimeSlot` logic. Co-authored SRS v2 and Checkpoint 1 Report.	Application main menu, execution commands, `Scheduler` structure, and time slot representation.
Semih Mertcan İÇYER	Implemented domain entities (`Course`, `Session`, `Room`, `Instructor`) and `FileDataReader` skeleton. Drawn UML Class Diagram and Flowchart for SDD v2.	UML Class Diagram, 11-step algorithm flowchart, domain model relationships, and constraint checking rules.

8. Development Plan to Checkpoint 2 (Weeks 8–10)

Version	Planned Technical Tasks	Requirements & Constraints	Task Owner
Code v2 (Week 8)	Implement file parsing in `FileDataReader` for courses, rooms, instructors, and requests. Log parse warnings.	FR-01 to FR-05, HC-05, HC-06, HC-07	Semih Mertcan İÇYER
Code v3 (Week 9)	Implement core rule-based scheduling algorithm loops, fixed-time request processing, and basic hard constraint checking.	FR-06 to FR-10, HC-01 to HC-04	Eren KARAKAŞ
Code v4 (Week 10)	Deliver working core automatic scheduling functionality, 5x8 year-group grid display, unscheduled reports (Checkpoint 2).	FR-11 to FR-14, Screen 3 & 4	Both Members

9. Risks and Mitigation Strategies

Identified Risk: Complexity in handling complex multi-constraint combinations during peak slot occupancy could lead to false unscheduled reports.
Mitigation Plan: Thoroughly test candidate ordering logic using the provided sample dataset and implement clean fallback checks in `ConstraintChecker` prior to Week 10.

AI-Use Declaration

Tool	Used For	Verification Method
ChatGPT / Claude / Gemini	Assisting in document formatting, consistency matrix layout, and report structure validation.	Both members verified all table entries and confirm 100% design-code consistency with `src/` files.